

5-step Latent Resfusion から Stable Diffusion への直接 handover

2026 年 8 月 17 日

1 目的

構成は次のとおりとする。

$x \longrightarrow \text{VAE encoder} \longrightarrow z_0 \longrightarrow \text{ADJSCC encoder}$
 $\longrightarrow \text{wireless channel} \longrightarrow \text{ADJSCC decoder} \longrightarrow \hat{z}_0 \longrightarrow \text{5-step Latent Resfusion}$
 $\longrightarrow \text{Stable Diffusion} \longrightarrow \text{VAE decoder}.$

Resfusion は原論文 [1] と公式実装 [2] どちらの residual noise だけを学習する。追加の residual head, bridge network, score loss, 統計 loss, 画像 loss は使わない。

推論時には、Resfusion を 0 回から 5 回まで実行した各状態を Stable Diffusion へ直接渡せるようにする。ここで「0 回」は Resfusion の初期状態をそのまま渡す場合、「5 回」は 5 回すべて denoise した後を表す。

2 学習用 latent pair

画像 x から Stable Diffusion の scaled VAE latent を作る。

$$z_0 = s E_{\text{VAE}}(x), \quad s = 0.18215.$$

この latent を ADJSCC で伝送する。

$$\begin{aligned} u &= E_{\text{JSCC}}(z_0), \\ y &= \text{Channel}(u), \\ \hat{z}_0 &= D_{\text{JSCC}}(y). \end{aligned}$$

したがって、Resfusion の paired data は

$$(\text{degraded latent}, \text{clean latent}) = (\hat{z}_0, z_0)$$

であり、prior residual は

$$R = \hat{z}_0 - z_0 \tag{1}$$

となる。VAE と学習済み ADJSCC は固定し、Latent Resfusion だけを学習する。

scaled latent は RGB 画像ではないため、公式 RGB 実装にある $[0, 1] \rightarrow [-1, 1]$ 変換と $[-1, 1]$ clamp は行わない。

3 論文どおりの Resfusion 学習

3.1 forward process

$$\alpha_t = 1 - \beta_t, \quad \bar{\alpha}_t = \prod_{i=0}^t \alpha_i.$$

以下では、質問中の α_t を cumulative alpha $\bar{\alpha}_t$ として表記する.

Resfusion の forward state を

$$v_t = \sqrt{\bar{\alpha}_t} z_0 + (1 - \sqrt{\bar{\alpha}_t}) R + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I) \quad (2)$$

とする. 式 (1) を代入すると

$$v_t = (2\sqrt{\bar{\alpha}_t} - 1) z_0 + (1 - \sqrt{\bar{\alpha}_t}) \hat{z}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon \quad (3)$$

となる.

3.2 学習 target

Resfusion U-Net が予測する residual noise は

$$\eta_t = \epsilon + \frac{(1 - \sqrt{\bar{\alpha}_t}) \sqrt{1 - \bar{\alpha}_t}}{\beta_t} R \quad (4)$$

である.

各 batch で

$$t \sim \text{Uniform}\{0, 1, 2, 3, 4\}, \quad \epsilon \sim \mathcal{N}(0, I)$$

を sample し, 式 (2) で v_t を作る. network への入力

$$(v_t, \hat{z}_0, t)$$

だけであり, 4 チャンルの v_t と 4 チャンルの $\hat{z}_0$ を結合するため, U-Net の入力は 8 チャンネル, 出力は 4 チャンネルとする.

3.3 loss

学習 loss は原論文の MSE だけとする.

$$\mathcal{L}_{\text{Resfusion}} = \mathbb{E}_{z_0, \hat{z}_0, t, \epsilon} \left[\|\eta_\theta(v_t, \hat{z}_0, t) - \eta_t\|_2^2 \right] \quad (5)$$

使用しないものは次のとおりである.

$$\mathcal{L}_R, \quad \mathcal{L}_{\text{bridge}}, \quad \mathcal{L}_{\text{score}}, \quad \mathcal{L}_{\text{stat}}, \quad \mathcal{L}_{\text{image}}.$$

また, SNR 専用 embedding や追加 head も設けない. channel 状態の違いは degraded condition $\hat{z}_0$ 自体から network へ伝わる.

4 5-step Resfusion

4.1 truncated schedule

公式復元設定と同じ

$$T = 12, \quad \text{LinearProScheduler}$$

を使用する. truncated strategy により

$$\sqrt{\bar{\alpha}_{t_*}} = \frac{1}{2}$$

となる点で schedule が切られ, 実際に使用する時刻は

$$t_R = 4, 3, 2, 1, 0$$

の 5 個になる. したがって U-Net 評価は 5 回である.

Resfusion 初期状態は

$$\begin{aligned} v_4 &= \sqrt{\bar{\alpha}_4^R} \hat{z}_0 + \sqrt{1 - \bar{\alpha}_4^R} \epsilon \\ &= 0.5 \hat{z}_0 + \sqrt{0.75} \epsilon \end{aligned} \tag{6}$$

とする.

各 reverse step は

$$v_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(v_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \eta_\theta(v_t, \hat{z}_0, t) \right) + \sqrt{\tilde{\beta}_t} \xi, \quad \xi \sim \mathcal{N}(0, I) \tag{7}$$

である. 最後の step では $\xi = 0$ とする.

4.2 0-5 step の定義

Resfusion を完了した回数を k とする.

$$\begin{aligned} k = 0 & : v_4 \quad (\text{初期化直後}), \\ k = 1 & : v_3 \quad (t_R = 4 \text{ を 1 回 denoise 後}), \\ k = 2 & : v_2, \\ k = 3 & : v_1, \\ k = 4 & : v_0, \\ k = 5 & : v_{-1} \quad (5 \text{ 回 denoise 後の clean 推定}). \end{aligned}$$

この 6 状態のいずれからでも Stable Diffusion へ handover する.

5 Stable Diffusion timestep との対応

5.1 対応規則

Resfusion と Stable Diffusion では noise schedule が異なるため, Resfusion の step 番号をそのまま Stable Diffusion へ渡してはいけない.

質問中の

$$(1 - \sqrt{\bar{\alpha}_t}) \hat{z}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$$

に現れる係数を noise level の識別に使う。Resfusion 状態 k と Stable Diffusion raw timestep τ の距離を

$$d(k, \tau) = \left[\left(1 - \sqrt{\bar{\alpha}_k^R} \right) - \left(1 - \sqrt{\bar{\alpha}_\tau^{\text{SD}}} \right) \right]^2 + \left[\sqrt{1 - \bar{\alpha}_k^R} - \sqrt{1 - \bar{\alpha}_\tau^{\text{SD}}} \right]^2 \quad (8)$$

とし,

$$\tau_k = \arg \min_{\tau \in \{0, \dots, 999\}} d(k, \tau) \quad (9)$$

で Stable Diffusion 側の時刻を選ぶ。

これは, signal/noise 比

$$\rho_t = \frac{\sqrt{1 - \bar{\alpha}_t}}{\sqrt{\bar{\alpha}_t}}$$

が最も近い時刻を選ぶことと実質的に同じである。

5.2 本プロジェクトでの対応表

Stable Diffusion v1 の

$$T_{\text{SD}} = 1000, \quad \text{linear_start} = 0.00085, \quad \text{linear_end} = 0.012$$

を用いる。ldm/modules/diffusionmodules/util.py と同じく, 端点の平方根を線形補間してから二乗した β_τ^{SD} を使う。

表 1 Resfusion の 0–5 denoise 回数と Stable Diffusion raw timestep

完了回数 k	Resfusion state	$\bar{\alpha}^R$	$\sqrt{\bar{\alpha}^R}$	$\sqrt{1 - \bar{\alpha}^R}$	SD raw τ_k
0	v_4	0.250000	0.500000	0.866025	520
1	v_3	0.310431	0.557163	0.830403	475
2	v_2	0.575518	0.758629	0.651523	309
3	v_1	0.833902	0.913182	0.407552	146
4	v_0	0.991667	0.995825	0.091287	9
5	v_{-1}	1.000000	1.000000	0.000000	0

よって handover 先は

$$k = 0, 1, 2, 3, 4, 5 \quad \longrightarrow \quad \tau_k = 520, 475, 309, 146, 9, 0$$

となる。

$k = 5$ は noise 0 であるが, Stable Diffusion の既存 schedule には厳密な noise 0 の raw timestep がない。そのため最も近い $\tau = 0$ を使う。

6 直接 handover 推論

handover では補正, re-noise, residual subtraction を一切行わない. Resfusion の現在状態をそのまま Stable Diffusion latent として使う.

$$\boxed{z_{\tau_k}^{\text{SD}} \leftarrow v^{(k)}} \quad (10)$$

ここで $v^{(k)}$ は Resfusion を k 回 denoise した状態である.

推論手順は次のとおりである.

1. ADJSCC decoder から $\hat{z}_0$ を得る.
2. 式 (6) で v_4 を作る.
3. 選択した k 回だけ式 (7) を実行する.
4. 表 1 から τ_k を得る.
5. 式 (10) により current state を直接 Stable Diffusion へ渡す.
6. Stable Diffusion を raw timestep τ_k から 0 まで実行する.
7. 得られた scaled latent を VAE decoder で画像へ戻す.

6.1 疑似コード

```
z_hat0 = adjsc_decode(received_signal)

v = 0.5 * z_hat0 + sqrt(0.75) * randn_like(z_hat0)

sd_timestep = [520, 475, 309, 146, 9, 0]

for k in range(6):
    if k == selected_handover_step:
        z_out = stable_diffusion_reverse(
            latent=v,
            start_raw_timestep=sd_timestep[k]
        )
        image_out = vae_decode_scaled(z_out)
        break

# k=0,...,4 のときだけ次の Resfusion step を実行
t_resfusion = 4 - k
eta_hat = resfusion_unet(v, z_hat0, t_resfusion)
v = resfusion_reverse_step(v, eta_hat, t_resfusion)
```

7 最小構成のまとめ

- Resfusion は公式設定と同じ 5 回の denoise を行う.
- 学習 target は residual noise η_t だけである.
- loss は式 (5) の MSE だけである.
- U-Net は 8 チャンネル入力, 4 チャンネル出力である.
- 0-5 回の各状態を直接 Stable Diffusion へ渡せる.
- 対応 raw timestep は 520, 475, 309, 146, 9, 0 である.
- handover 時に tensor の補正や追加 noise は行わない.

参考文献

- [1] Z. Shi et al., “Resfusion: Denoising Diffusion Probabilistic Models for Image Restoration Based on Prior Residual Noise,” *NeurIPS 2024*. <https://arxiv.org/abs/2311.14900>
- [2] NKICSL, “Official implementation of Resfusion.” <https://github.com/nkicsl/Resfusion>